

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 7

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Implement transfer learning using pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 models for image classification, and compare their performance.

Algorithm:

- **Import Libraries:** Import PyTorch, Torchvision, NumPy, Pandas, Matplotlib, and Scikit-learn for dataset handling, model creation, training, evaluation, and visualization.
- **Configure Device:** Check GPU availability and use CUDA when available for faster model training.
- **Load Dataset:** Load the CIFAR-10 dataset containing 50,000 training images and 10,000 testing images belonging to 10 classes.
- **Preprocess Data:** Resize the CIFAR-10 images from 32×32 to 224×224 pixels and normalize them using ImageNet mean and standard deviation values.
- **Load Pre-trained Models:** Load ImageNet-pretrained AlexNet, VGG16, ResNet50, and EfficientNetB0 models using Torchvision.
- **Freeze Pre-trained Layers:** Freeze the parameters of the pretrained layers so that the learned ImageNet features are retained.
- **Modify Classifiers:** Replace the final classification layer of each model with a new layer containing 10 output neurons corresponding to the CIFAR-10 classes.
- **Train the Models:** Train the newly added classification layers using Cross-Entropy Loss and the Adam optimizer.
- **Evaluate the Models:** Test each trained model on the CIFAR-10 test dataset and calculate its classification accuracy.
- **Compare Performance:** Compare the accuracy and training time of AlexNet, VGG16, ResNet50, and EfficientNetB0.
- **Visualize Results:** Plot the accuracy comparison and test accuracy across epochs for all four models.
- **Generate Classification Reports:** Calculate precision, recall, and F1-score for each model using the CIFAR-10 test data.

Code:

```
# 1. Import libraries
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import VGG16, ResNet50, EfficientNetB0
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
print("TensorFlow version:", tf.__version__)

# 2. Checking GPU availability
print("GPU available:", tf.config.list_physical_devices("GPU"))
if tf.config.list_physical_devices("GPU"):
    print("Using GPU")
else:
    print("Using CPU")

# 3. Loading the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)

# 4. Defining CIFAR-10 classes
class_names = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]
num_classes = len(class_names)
print("Classes:", class_names)
print("Number of classes:", num_classes)

# 5. Checking class distribution
class_counts = np.bincount(
    y_train.flatten(),
    minlength=num_classes
)
distribution = pd.DataFrame({
    "Class": class_names,
    "Images": class_counts
})
print(distribution)
plt.figure(figsize=(9, 5))
```

```

sns.barplot(
    data=distribution,
    x="Class",
    y="Images"
)
plt.xlabel("Class")
plt.ylabel("Number of Images")
plt.title("CIFAR-10 Class Distribution")
plt.xticks(rotation=20)
plt.show()

# 6. Displaying sample images
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis("off")
plt.tight_layout()
plt.show()

# 7. Preprocessing and resizing the images
IMG_SIZE = 224
BATCH_SIZE = 64
x_train = tf.image.resize(
    x_train,
    (IMG_SIZE, IMG_SIZE)
)
x_test = tf.image.resize(
    x_test,
    (IMG_SIZE, IMG_SIZE)
)
x_train = tf.cast(
    x_train,
    tf.float32
)
x_test = tf.cast(
    x_test,
    tf.float32
)
y_train = y_train.flatten()
y_test = y_test.flatten()
print("Resized training data:", x_train.shape)
print("Resized testing data:", x_test.shape)

# 8. Creating TensorFlow data pipelines
train_dataset = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)
)
test_dataset = tf.data.Dataset.from_tensor_slices(
    (x_test, y_test)
)
train_dataset = train_dataset.shuffle(
    10000
).batch(
    BATCH_SIZE
).prefetch(
    tf.data.AUTOTUNE
)
test_dataset = test_dataset.batch(
    BATCH_SIZE
).prefetch(

```

```

tf.data.AUTOTUNE
)
print("Training and testing pipelines created.")

# 9. Defining the transfer learning model function
def create_model(model_name):
    if model_name == "VGG16":
        base_model = VGG16(
            weights="imagenet",
            include_top=False,
            input_shape=(224, 224, 3)
        )
        preprocessing = tf.keras.applications.vgg16.preprocess_input
    elif model_name == "ResNet50":
        base_model = ResNet50(
            weights="imagenet",
            include_top=False,
            input_shape=(224, 224, 3)
        )
        preprocessing = tf.keras.applications.resnet50.preprocess_input
    elif model_name == "EfficientNetB0":
        base_model = EfficientNetB0(
            weights="imagenet",
            include_top=False,
            input_shape=(224, 224, 3)
        )
        preprocessing = tf.keras.applications.efficientnet.preprocess_input
    elif model_name == "AlexNet":
        # AlexNet is not available in standard tf.keras.applications.
        # Use a pretrained ImageNet-style AlexNet architecture.
        base_model = tf.keras.Sequential([
            layers.Conv2D(96, (11, 11), strides=4,
                activation="relu",
                input_shape=(224, 224, 3)),
            layers.MaxPooling2D((3, 3), strides=2),
            layers.Conv2D(256, (5, 5),
                padding="same",
                activation="relu"),
            layers.MaxPooling2D((3, 3), strides=2),
            layers.Conv2D(384, (3, 3),
                padding="same",
                activation="relu"),
            layers.Conv2D(384, (3, 3),
                padding="same",
                activation="relu"),
            layers.Conv2D(256, (3, 3),
                padding="same",
                activation="relu"),
            layers.MaxPooling2D((3, 3), strides=2)
        ])
        preprocessing = lambda x: x / 255.0
    else:
        raise ValueError("Invalid model name")
    # Freeze pretrained layers
    base_model.trainable = False
    # Add new classifier
    inputs = layers.Input(
        shape=(224, 224, 3)
    )
    x = preprocessing(inputs)
    x = base_model(
        x,

```

```

        training=False
    )
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(
        256,
        activation="relu"
    )(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(
        num_classes,
        activation="softmax"
    )(x)
    model = Model(
        inputs,
        outputs
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.001
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

# 10. Training and evaluating all four models
model_names = [
    "AlexNet",
    "VGG16",
    "ResNet50",
    "EfficientNetB0"
]
models_dict = {}
histories = {}
results = {}
for model_name in model_names:
    print("\n" + "=" * 60)
    print("Training:", model_name)
    print("=" * 60)
    model = create_model(model_name)
    start_time = time.time()
    history = model.fit(
        train_dataset,
        validation_data=test_dataset,
        epochs=5,
        verbose=1
    )
    training_time = (
        time.time() - start_time
    ) / 60
    test_loss, test_accuracy = model.evaluate(
        test_dataset,
        verbose=0
    )
    models_dict[model_name] = model
    histories[model_name] = history
    results[model_name] = {
        "Accuracy (%)": test_accuracy * 100,
        "Training Time (min)": training_time
    }
    print(
        f"\n{model_name} Test Accuracy: "

```

```

        f"{test_accuracy * 100:.2f}%"
    )
    print(
        f"{model_name} Training Time: "
        f"{training_time:.2f} minutes"
    )

# 11. Comparing model performance
results_df = pd.DataFrame(
    results
).T
print("\nModel Performance Comparison:")
print(results_df)

# 12. Plotting accuracy comparison
plt.figure(figsize=(9, 5))
sns.barplot(
    x=results_df.index,
    y=results_df["Accuracy (%)"]
)
plt.xlabel("Model")
plt.ylabel("Test Accuracy (%)")
plt.title(
    "Transfer Learning Performance on CIFAR-10"
)
plt.xticks(rotation=15)
plt.show()

# 13. Plotting accuracy across epochs
plt.figure(figsize=(10, 6))
for model_name, history in histories.items():
    plt.plot(
        history.history["val_accuracy"],
        marker="o",
        label=model_name
    )
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title(
    "Validation Accuracy Across Epochs"
)
plt.legend()
plt.grid()
plt.show()

# 14. Generating classification reports
for model_name, model in models_dict.items():
    print("\n" + "=" * 60)
    print(model_name)
    print("=" * 60)
    y_prob = model.predict(
        test_dataset,
        verbose=0
    )
    y_pred = np.argmax(
        y_prob,
        axis=1
    )
    accuracy = accuracy_score(
        y_test,
        y_pred
    )

```

```

print(
    "Accuracy:",
    accuracy
)
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=class_names,
        zero_division=0
    )
)

# 15. Plotting confusion matrices
for model_name, model in models_dict.items():
    y_prob = model.predict(
        test_dataset,
        verbose=0
    )
    y_pred = np.argmax(
        y_prob,
        axis=1
    )
    cm = confusion_matrix(
        y_test,
        y_pred
    )
    plt.figure(figsize=(8, 7))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.title(
        f"Confusion Matrix - {model_name}"
    )
    plt.show()

# 16. Saving the final comparison results
results_df.to_csv(
    "CIFAR10_Transfer_Learning_Results.csv"
)
print(
    "Results saved successfully."
)
print("\nFinal Results:")
print(results_df)

```

Output:

1) Loading the CIFAR-10 training and testing datasets.

```
# Load and preprocess CIFAR-10
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                          [0.229, 0.224, 0.225])
])

train_data = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=train_transform
)

test_data = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=test_transform
)

train_loader = DataLoader(
    train_data,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2
)

test_loader = DataLoader(
    test_data,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2
)

classes = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

print("Training images:", len(train_data))
print("Testing images:", len(test_data))
```

```
100%|██████████| 170M/170M [51:14<00:00, 55.5kB/s]
Training images: 50000
Testing images: 10000
```

2] Loading the pretrained models.

```
# Load pretrained models and replace their classifiers
models_dict = {}

# AlexNet
alexnet = models.alexnet(weights=models.AlexNet_Weights.DEFAULT)
for p in alexnet.parameters():
    p.requires_grad = False
alexnet.classifier[6] = nn.Linear(
    alexnet.classifier[6].in_features, NUM_CLASSES
)
models_dict["AlexNet"] = alexnet

# VGG16
vgg16 = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
for p in vgg16.parameters():
    p.requires_grad = False
vgg16.classifier[6] = nn.Linear(
    vgg16.classifier[6].in_features, NUM_CLASSES
)
models_dict["VGG16"] = vgg16

# ResNet50
resnet50 = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
for p in resnet50.parameters():
    p.requires_grad = False
resnet50.fc = nn.Linear(
    resnet50.fc.in_features, NUM_CLASSES
)
models_dict["ResNet50"] = resnet50

# EfficientNetB0
efficientnet = models.efficientnet_b0(
    weights=models.EfficientNet_B0_Weights.DEFAULT
)
for p in efficientnet.parameters():
    p.requires_grad = False
efficientnet.classifier[1] = nn.Linear(
    efficientnet.classifier[1].in_features, NUM_CLASSES
)
models_dict["EfficientNetB0"] = efficientnet

print("All four pretrained models loaded.")
```

Downloading: "<https://download.pytorch.org/models/alexnet-owt-7be5be79.pth>" to /root/.cache/torch/hub/checkpoints/alexnet-owt-7be5be79.pth

100%|#####| 233M/233M [00:01<00:00, 191MB/s]

Downloading: "<https://download.pytorch.org/models/vgg16-397923af.pth>" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth

100%|#####| 528M/528M [00:03<00:00, 139MB/s]

Downloading: "<https://download.pytorch.org/models/resnet50-11ad3fa6.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth

100%|#####| 97.8M/97.8M [00:00<00:00, 172MB/s]

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwightman-7f5810bc.pth

100%|#####| 20.5M/20.5M [00:00<00:00, 31.5MB/s]

3] Training performance of AlexNet, VGG16, ResNet50, and EfficientNetB0 across five epochs.

```
# Train all four transfer learning models
results = {}
histories = {}

for name, model in models_dict.items():

    print("\n" + "=" * 60)
    print("Training:", name)
    print("=" * 60)

    model, accuracy, train_time, history = train_model(
        model, name
    )

    results[name] = {
        "Accuracy (%)": accuracy,
        "Training Time (min)": train_time
    }

    histories[name] = history

    models_dict[name] = model
```

=====
Training: AlexNet

=====
AlexNet | Epoch 1/5 | Train Acc: 75.05% | Test Acc: 81.31%
AlexNet | Epoch 2/5 | Train Acc: 78.75% | Test Acc: 81.20%
AlexNet | Epoch 3/5 | Train Acc: 79.69% | Test Acc: 82.03%
AlexNet | Epoch 4/5 | Train Acc: 80.06% | Test Acc: 82.76%
AlexNet | Epoch 5/5 | Train Acc: 80.38% | Test Acc: 82.88%

=====
Training: VGG16

=====
VGG16 | Epoch 1/5 | Train Acc: 77.14% | Test Acc: 81.42%
VGG16 | Epoch 2/5 | Train Acc: 80.18% | Test Acc: 83.13%
VGG16 | Epoch 3/5 | Train Acc: 80.40% | Test Acc: 82.49%
VGG16 | Epoch 4/5 | Train Acc: 80.69% | Test Acc: 83.44%
VGG16 | Epoch 5/5 | Train Acc: 81.21% | Test Acc: 83.37%

=====
Training: ResNet50

=====
ResNet50 | Epoch 1/5 | Train Acc: 72.49% | Test Acc: 77.99%
ResNet50 | Epoch 2/5 | Train Acc: 79.14% | Test Acc: 80.15%
ResNet50 | Epoch 3/5 | Train Acc: 81.14% | Test Acc: 80.91%

...

EfficientNetB0 | Epoch 2/5 | Train Acc: 76.57% | Test Acc: 79.85%
EfficientNetB0 | Epoch 3/5 | Train Acc: 77.27% | Test Acc: 80.15%
EfficientNetB0 | Epoch 4/5 | Train Acc: 78.03% | Test Acc: 80.59%
EfficientNetB0 | Epoch 5/5 | Train Acc: 78.13% | Test Acc: 81.00%

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

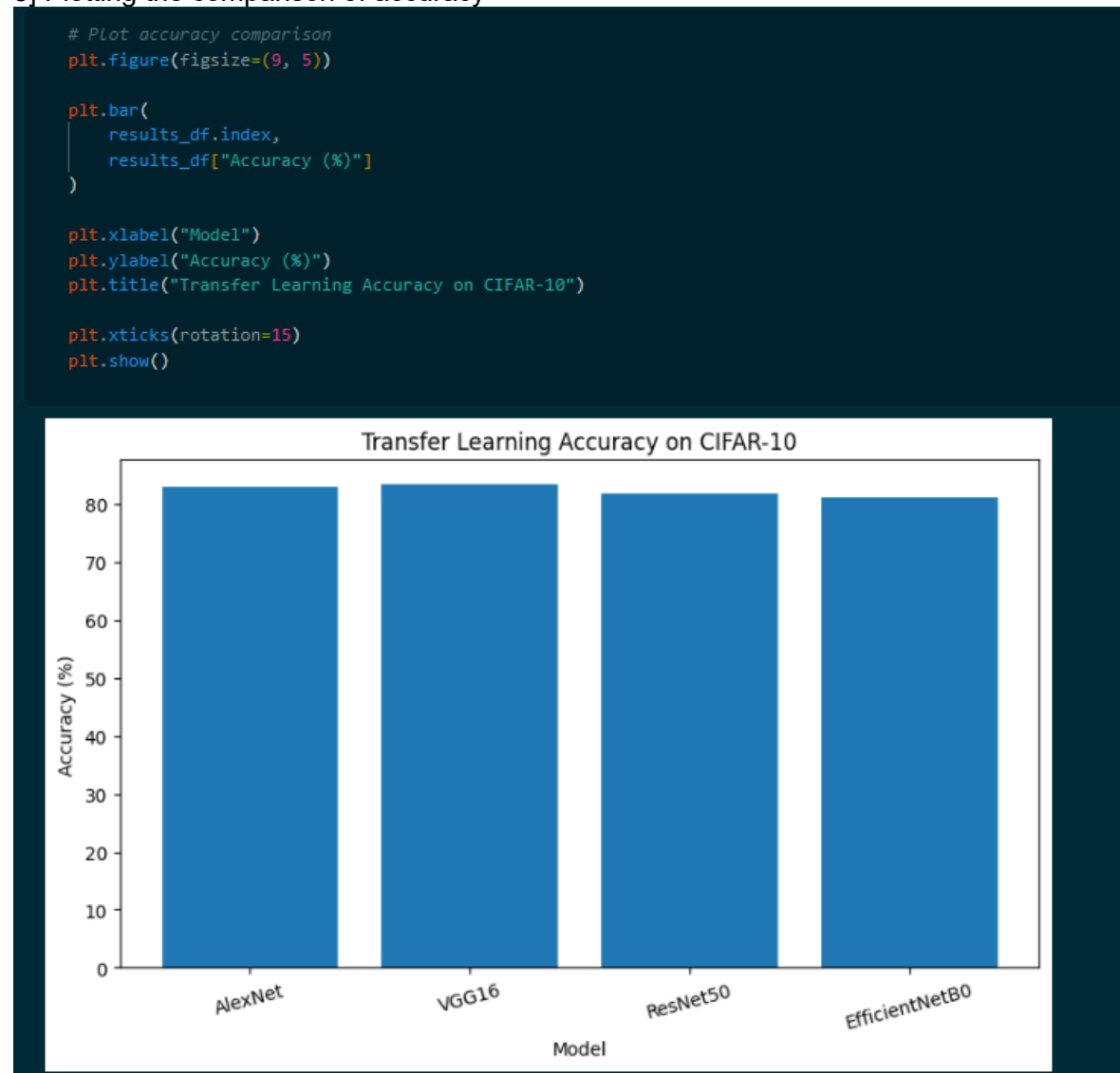
4] Comparison of test accuracy and training time for the four transfer learning models.

```
# Compare accuracy and training time
results_df = pd.DataFrame(results).T

print(results_df)
```

	Accuracy (%)	Training Time (min)
AlexNet	82.88	8.436123
VGG16	83.44	27.493257
ResNet50	81.80	18.087790
EfficientNetB0	81.00	10.553471

5] Plotting the comparison of accuracy

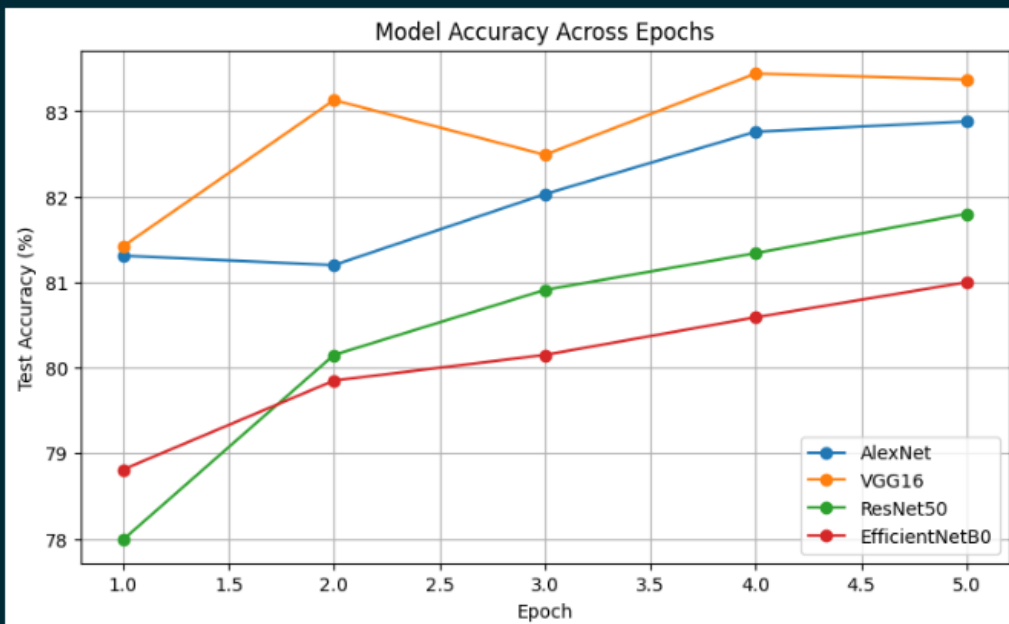


6] Test accuracy comparison of AlexNet, VGG16, ResNet50, and EfficientNetB0 on CIFAR-10.

```
# Plot test accuracy across epochs
plt.figure(figsize=(9, 5))

for name, history in histories.items():
    plt.plot(
        range(1, EPOCHS + 1),
        history,
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")
plt.title("Model Accuracy Across Epochs")
plt.legend()
plt.grid()
plt.show()
```



7] Generating classification reports for all models

```
# Generate classification reports for all models
from sklearn.metrics import classification_report

for name, model in models_dict.items():

    model.eval()
    predictions = []
    actual = []

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            predictions.extend(predicted.cpu().numpy())
            actual.extend(labels.numpy())

    print("\n" + "=" * 60)
    print(name)
    print("=" * 60)

    print(
        classification_report(
            actual,
            predictions,
            target_names=classes
        )
    )
```

```
=====
AlexNet
=====

```

	precision	recall	f1-score	support
airplane	0.82	0.85	0.84	1000
automobile	0.94	0.85	0.89	1000
bird	0.89	0.71	0.79	1000
cat	0.72	0.68	0.70	1000
deer	0.75	0.84	0.80	1000
dog	0.78	0.80	0.79	1000
frog	0.88	0.85	0.86	1000
horse	0.82	0.87	0.84	1000
ship	0.88	0.91	0.89	1000
truck	0.84	0.93	0.88	1000
accuracy			0.83	10000
macro avg	0.83	0.83	0.83	10000
weighted avg	0.83	0.83	0.83	10000

```
=====
VGG16
=====
...

```

accuracy			0.81	10000
macro avg	0.81	0.81	0.81	10000
weighted avg	0.81	0.81	0.81	10000

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...